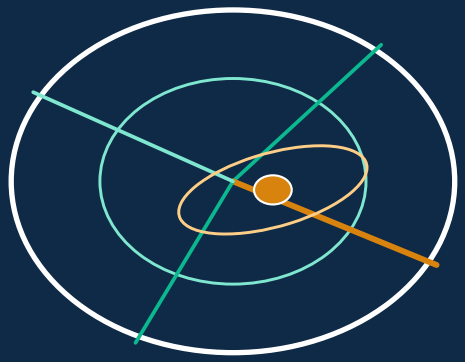
ARES ATAK

A hands-on tour of the most important features



- Running it · the map · coverage & point-to-point links
- Geolocation by line of bearing — the ML fix and its error ellipse
- The SDR console — single- vs multi-channel, GPS, compass modes & calibration
- The DF panel — stacked spectrum + waterfall + compass + options
- Distributed sensing over a MANET · group chat · the ATAK / Server console
- HF & satellites · security · where to learn more

Open `http://localhost:8000` (or :3000) alongside this deck — the interactive API is at `/docs`.

Ares ATAK v2.0 · regenerate with `docs/build_pdfs.py`

Architecture — what's under the hood

Three pieces talking over plain web protocols

Backend — Python (FastAPI · NumPy)

The physics engine — ITS Longley-Rice (ITM) + ~12 models, diffraction, atmosphere, HF (ITU-R-style), SGP4 satellites. The DF solver — ML triangulation + covariance ellipse, TDOA/FDOA, phase interferometry / MUSIC, EKF tracks. The SDR manager, the CoT-to-ATAK bridge, the MANET peer mesh + group chat. ~90 REST endpoints + a WebSocket.

→ **backend/app/**

REST + WebSocket

(JSON over HTTP)

Frontend — JavaScript / React (Vite · Cesium)

The screen — the 2-D Leaflet map + the 3-D CesiumJS globe, the bottom-panel tabs (Results / Terrain / 3-D / DF / Chat / Layers / …), the SDR and ATAK-Server consoles, the drawing tools and NATO symbology. Talks to the backend over REST + a live WebSocket. Packaged with Electron as a desktop app.

→ **frontend/src/**

ATAK plugin — Kotlin (Android)

SOOTHSAYER-style parity against an Ares server, plus DF. SDK-blocked for build/signing — it needs the tak.gov SDK and a publisher account.

→ **atak-plugin/**

Glue — git · Docker · CI · the shell

start-backend.sh / start-web.sh / start-desktop.sh · install.sh · docker-compose.yml · .github/workflows/ci.yml (the 53-check backend harness + the 8-check frontend tests + a bundle/build).

Running it — server, UI, desktop, Docker

- **Backend (the engine).**
  cd ares-authoritative && ./start-backend.sh — runs FastAPI on :8000. The first run with auth on logs an admin password once (auth is ON by default unless the server is bound to a loopback address).
- **Web UI.**
  ./start-web.sh — backend + a built UI on :3000, opens a browser. For development: cd frontend && npm install && npm run dev (:5173).
- **Desktop app.**
  ./start-desktop.sh — the same UI wrapped in Electron.
- **Docker.**
  docker compose up -d — backend + frontend in containers; point ARES_PACKS_HOST_DIR at a pre-staged packs disk, set ARES_AUTH / ARES_NETWORK_POLICY / ARES_MESH_SECRET as needed.
- **Air-gapped.**
  ./install.sh --offline-bundle <dir> stages a pre-built data bundle and skips the online terrain download; then run with ARES_NETWORK_POLICY=offline_only.
- **Poke the API directly.**
  http://localhost:8000/docs — every endpoint, with a 'Try it out' button. The fastest way to learn what the engine actually does.

The window, once it's up

Header: the mode (Propagation / Geolocation), the ATAK / Server console, the SDR console, a GPU badge, Run. Left column: the transmitter / antenna / atmosphere / propagation controls. Centre: the map (2-D ⇄ 3-D toggle in the ⚙ menu).

Bottom panel (collapsible): Results · Terrain Profile · Link Budget · 3-D View · DF · Chat · dB Calc · Layers · Emitter Summary · Saved Locations · Space Wx. Right-click the map to drop an emitter.

The map — 2-D Leaflet ⇄ 3-D CesiumJS globe

- **2-D ⇄ 3-D.**
  Leaflet 2-D map by default; the ⚙ menu (top-right of the map) or the bottom-panel '3-D View' tab switches to the CesiumJS globe. Camera, layers, drawn features, KMZ and your GPS marker carry across — KMZ added in 2-D persists in 3-D and back.
- **Layers.**
  Drag-and-drop KML / KMZ / GeoJSON / GPX / GeoTIFF / DTED / images onto the map → they appear in the Layers tab (toggle, recolour, remove) and on both views. The import button on the globe imports the same way.
- **Drawing tools.**
  The ✎ palette — points, lines, polygons, rectangles, circles, ellipses, freehand, range rings, fans, range-bearing, geofences, MIL / NATO symbology. (Its z-index was fixed so the bottom panel no longer hides it.)
- **Ruler & search.**
  The ruler tool gives two-click distance/bearing; the search box does place search (Nominatim); ⊕ re-centres on the transmitter.
- **Settings (⚙).**
  Basemap, distance/altitude units, coordinate system, compass rose, brightness, the 3-D coverage render mode (heatmap vs point-cloud), feature colours — shared by both views.
- **On the map you'll see.**
  the TX (cyan) and RX (yellow) markers, coverage heatmaps, LoB fans, error ellipses and suspected-emitter markers, antenna lobes, KMZ overlays, and a cyan '▲ you' GPS marker.

Coverage simulation — place, model, run

- **1. Place the transmitter.**
  Right-click the map (or set lat/lon in the left panel). Set height AGL, power, frequency, and the antenna — type / gain / azimuth / tilt, a polar pattern (omni · sector · Yagi · dish · …) or an imported measured pattern.
- **2. Pick a propagation model.**
  ITM (Longley-Rice — the default, terrain-aware) · Okumura-Hata urban/suburban/rural · COST-231 · two-ray · ITU-R P.1546 / P.528 · Egli / Ericsson / SUI · radar. Or 'auto-select' suggests one from frequency / range / environment. Add a diffraction method (Deygout / Bullington / …) and the environment / clutter.
- **3. (Optional) raster mode.**
  Tick 'raster' next to Run — one ITM path per grid cell instead of a radial sweep: even coverage everywhere, no thinning at long range (heavier — pick it when you want a uniform raster).
- **4. Run.**
  The header's Run button. Progress streams over a WebSocket; the result is a colour-graded coverage layer (signal strength) on the 2-D map and the 3-D globe. The Results / Link-Budget tabs show the numbers; Space-Wx shows any HF corrections applied.
- **Also under 'simulate'.**
  multisite (all radios fused) · best-server · best-site / best-site-polygon (Monte-Carlo candidate ranking) · interference / EMCON · ray-trace · route · MANET coverage · satellite visibility.

Point-to-point links & terrain profiles

- **Set the receiver.**
  Switch the Coverage tab to 'P2P' (point-to-point); click the map to drop the RX, set its height (and altitude for an airborne RX). The TX/RX pair shows a dashed link line, a translucent first-Fresnel ellipsoid along it on the globe, and an antenna lobe at the TX.

- **Run the link.**
  Run computes the full link budget along the real terrain profile — path loss, received signal vs. sensitivity, fade margin, propagation mode (LOS / diffraction / troposcatter) and the radio horizons.

- **Terrain profile & obstruction.**
  The 'Terrain Profile' bottom tab shows the path's elevation cross-section with the line-of-sight, the Fresnel zone, and where (if anywhere) terrain blocks it. On the globe a red marker is dropped on the blocking ridge with the clearance deficit (a 4/3-earth check).

- **Note on the engine.**
  P2P uses the same ITS Longley-Rice port as coverage — the model SPLAT! / Radio Mobile / the FCC use — not a simplified approximation. The LOS↔transhorizon mode label keys on the terrain take-off angles, so a deep mid-path ridge is correctly labelled 'diffraction'.

Geolocation — line-of-bearing → ML fix

- **Switch to Geolocation mode.**
  Header → mode → Geolocation. The map gets a DF workflow; the 'Emitter Summary' bottom tab tracks the picture.
- **Add lines of bearing.**
  Radial-menu 'Add LoB from here' on a self/sensor marker (or the LoB list panel): enter the azimuth, RSSI, frequency, the receiving antenna pattern, observer height, a confidence, and an emitter id (DMR / IMSI / MAC / callsign) and timestamp if known. Each LoB draws as a bearing wedge.
- **The fix appears automatically.**
  Two LoBs at the same frequency → a 'Cut'; three or more → a 'Fix' — computed by a maximum-likelihood (iteratively-reweighted Gauss-Newton) triangulator. You get the emitter position, the covariance-derived error ellipse (it stretches along bad-geometry directions, like a real DF system's), GDOP, the residual RMS, and CEP / 95 % radii.
- **Terrain-aware bearings.**
  With 'terrain-aware' on, a bearing without an explicit range is capped by the propagation engine — a terrain radial finds where the modelled signal crosses the observed RSSI. DF that respects mountains.
- **Beyond bearings.**
  POST /api/v1/geolocate/multilaterate does TDOA (± FDOA) hyperbolic location from ≥3 receivers (Chan-style init + weighted Gauss-Newton). POST /api/v1/df/aoa turns an antenna-array snapshot (inter-channel phases, or IQ) into a bearing via phase interferometry / MUSIC / Capon, with the CRLB σ — then feeds it straight into the fix.

SDR console (1/2) — devices, GPS & compass

- **Open it.**
  The SDR console button in the header. It lists your SDR sources, the live DF picture, the GPS section, and a pointer to where the CoT push targets live (the ATAK / Server console).
- **Add a source — pick a class.**
  Single-channel: monitor a spectrum / decode audio, but it cannot produce a line of bearing (DF needs ≥2 coherent channels — the manager rejects its LoBs and tells you why). Multi-channel: declare the channel count (e.g. 5 for a KrakenSDR) and the array type (UCA / ULA / custom) + element spacing → it does DF.
- **Type & host.**
  KrakenSDR (polls krakensdr_doa's DOA_value over HTTP), Epiq Matchstiq X40 (an external DF process pushes JSON-lines), or generic JSON-lines TCP. Enter host:port, the device's lat/lon (or tick 'use GPS'), the frequency, and the active-signal threshold.
- **LoB accuracy estimate.**
  As you set the channel count / array, Ares shows the expected 1-σ bearing accuracy and the CEP-at-1-km (the interferometry CRLB plus a ~2.5° practical calibration floor that tightens with more channels).
- **GPS.**
  Set your position in the SDR console (or POST /api/v1/sdr/gps from gpsd / a phone) — it shows as the '▲ you' marker on both maps and becomes the observer position for LoBs that arrive without one.
- **Compass mode + calibration.**
  Absolute LOB (true north — map-plottable) · Relative LOB (degrees off the antenna front, 0° = front) · Clock position. Absolute = (0° + heading) + Relative. The DF panel has a guided calibrate form: aim the antenna at a target whose true bearing you know, shoot a LoB, enter both → heading = (true − relative).

SDR console (2/2) — the DF bottom-panel tab

The 'DF' bottom-panel tab — three columns

LEFT ≈ ½	MIDDLE	RIGHT
stacked spectrum viewer(s)	LoB compass	DF options

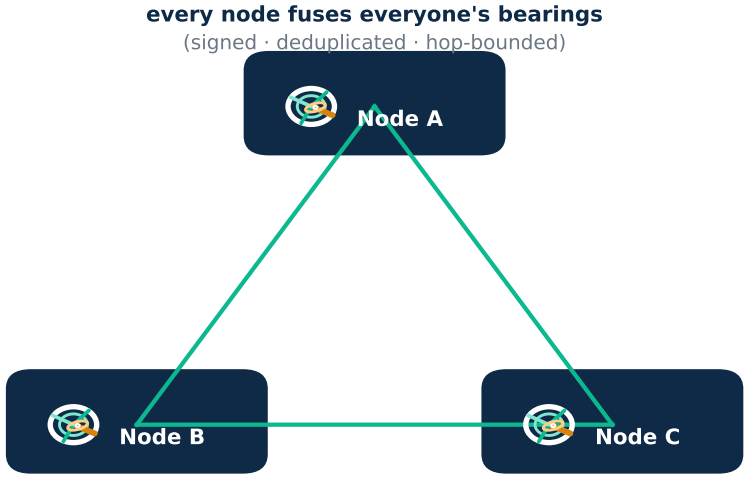
- **Left — spectrum viewer(s).**
  One viewer for a single-channel source; one per channel, vertically stacked, for a multi-channel array. Scroll-wheel zooms the frequency axis about the cursor; drag pans; click drops the DF tuner. The y-axis never moves — it's pinned to [noise floor − pad, peak + pad], so the noise floor and the strongest signal stay visible. Threshold / noise-floor / peak lines are drawn. A ▦ 'waterfall' toggle opens a scrolling spectrogram underneath each viewer — historical activity, bursts, hopping. (Synthetic until a SoapySDR / rtl-sdr / krakensdr-DAQ capture layer is wired — install SoapySDR and it goes live.)
- **Middle — LoB compass.**
  A true-north dial with a needle per current line of bearing; the freshest one labelled. In Relative / Clock mode it also draws the antenna-front reference and prints the latest LoB in all three reps — 'abs 117° · rel 12° · 4 o'clock'. Shows the active-LoB count, or 'single-channel — no LoBs'.
- **Right — DF options.**
  DF tune frequency (type, or drop the tuner on a signal) · threshold (min power for a bin to count as active → shoot a LoB) · gain · AGC · demodulate & listen (NFM/AM/SSB built-in; DMR / dPMR / P25 P1+P2 / TETRA / NXDN / D-STAR / YSF / M17 / POCSAG / … dispatched to an installed decoder — op25 / dsd-fme / sdrtrunk / tetra-rx / multimon-ng — it tells you if one isn't installed) · per-spectrum centre / span · the LoB-accuracy estimate + GPS status.

More channels ⇒ tighter LoBs. LoBs that arrive are plotted on the map automatically from your GPS location, fused with any others at the same frequency, and pushed to ATAK as CoT.

Distributed sensing over a MANET

Two ways to fuse more than one sensor

- **Same box.**
  Just register several SDRs under Devices — the solver groups LoBs by frequency across devices, so two/three antennas on one Ares produce a multi-sensor Cut/Fix automatically.
- **Over a MANET.**
  In the SDR console's 'Distributed sensing — mesh peers' section, add peer Ares nodes' base URLs (e.g. http://node2.lan:8000). Each node opens a WebSocket to every peer's stream and ingests their LoBs / fixes / chat; because it's symmetric, the union of every node's bearings is fused on every node — losing a node loses a sensor, not the picture.
- **Trust & integrity.**
  Set ARES_MESH_SECRET (or let Ares generate one in data/.mesh_secret the first time you add a peer — then copy it to the other nodes). Every inter-node LoB and chat message carries an HMAC-SHA256 over its content, so a node holding the secret rejects unsigned / tampered / origin-replayed peer data — a rogue node can't bias everyone's fixes. Loop-safe: dedup by (origin, id), hop-count TTL; messages propagate transitively, so even a partial mesh converges.
- **Where it shows.**
  fused fixes (with the n-LoBs and the contributing nodes) on the maps; the mesh status in the SDR console and at GET /api/v1/sdr/mesh; peer add/remove in the audit log.



Group chat — meshed, bridged to ATAK GeoChat

- **Open it.**
  The 'Chat' bottom-panel tab. Pick a room (channel — 'All' is the default; type a name to make a new one), set your callsign (remembered in the browser).
- **One conversation, three carriers.**
  A message broadcasts on this node's WebSocket, the peer mesh re-ingests it on every Ares node (HMAC-signed, deduplicated, hop-count-bounded), and it goes out as a CoT GeoChat — so ATAK / WinTAK clients on the bus see and can answer it. Inbound GeoChat from ATAK is routed back in by the CoT listener. It's the same chat across Ares nodes and ATAK.
- **Geo-tagged messages.**
  Tick the location toggle to attach your browser position; a received message with coordinates shows a clickable position (drops the RX marker there).
- **In the message list.**
  sender callsign · origin node (or '(ATAK)' / 'hop N') · time · text. Your own messages are right-aligned; ATAK ones are tinted.

Under the hood: app/core/chat.py · REST: GET /api/v1/chat/messages | /chat/rooms, POST /api/v1/chat/send · mesh: the same WebSocket the LoBs travel on · CoT type b-t-f (GeoChat), in and out.

The ATAK / Server console — packs, CoT, KMZ

- **Open it.**
  The ATAK / Server button in the header. It's the operator's offline-ops + TAK-integration panel.
- **Server status.**
  Version, online/offline, GPU, disk free, auth state. An 'ATAK integration: ON/OFF' toggle (the master switch for data packs / templates / KMZ export / CoT push). A loud security warning here if auth is off and the server isn't bound to loopback.
- **Cursor-on-Target push targets.**
  Where LoBs / fixes / GeoChat are sent — one per line: udp://239.2.3.1:6969 (the conventional ATAK multicast group), tcp://taksrv.lan:8087, or tls://taksrv.lan:8089 for mutual-TLS to a TAK Server (set ARES_COT_TLS_CA / CERT / KEY).
- **Offline data packs.**
  Per layer — terrain (SRTM30) · osm · imagery (ESRI World Imagery) · buildings (OSM/Overpass) · clutter (ESA WorldCover). A 'download region pack' form (bbox + max zoom), a job poller, a verify button (integrity / version), and delete. The 3-D globe renders installed packs — extruded buildings, offline imagery, real relief.
- **Radio templates & KMZ.**
  The radio templates the ATAK plugin would see (CRUD). On the map: import a KMZ/KML; export the current coverage as a KMZ GroundOverlay (an ATAK image-overlay / WinTAK / Google Earth).

HF circuits & satellites

- **HF circuit prediction.**
  GET /api/v1/hf/muf?lat1=&lon1=&lat2=&lon2=&freq_mhz= — an ITU-R-P.533-style model: multi-hop F2 geometry (hops, take-off angle, incidence at the F2 layer and the 110 km D-region), a parameterised foF2 (solar activity / zenith angle / geomagnetic latitude), the path MUF / FOT (= 0.85·MUF) / HPF / LUF via the secant law, ITU-R P.533 D-region absorption summed over hops, basic loss, received SNR vs an ITU-R P.372 noise floor, and a circuit-reliability percentage. If a voacapl / ITURHFPROP binary is on the PATH it's used instead. (foF2 is parameterised, not the CCIR/URSI coefficient maps — labelled honestly.)

- **Satellite visibility.**
  POST /api/v1/simulate/satellite_visibility {constellation, ground_lat, ground_lon, min_elevation_deg} — pulls TLEs from CelesTrak and propagates each with SGP4 (the canonical `sgp4` package if installed, else a vendored faithful near-earth SGP4, WGS-72): sub-points, footprint radii, and true topocentric az / el / slant-range to your ground station. Deep-space objects (period ≥ 225 min) are flagged with a 'pip install sgp4' hint for SDP4-grade accuracy.

- **Space weather.**
  GET /api/v1/space_weather — current NOAA SWPC indices (Kp / SFI, R/G storm classes), folded into the HF model and shown in the 'Space Wx' tab; degrades gracefully offline to last-known values.

Security & where to learn more

- **Auth.**
  ARES_AUTH = true | false | auto (the default). 'auto' ⇒ auth ON unless bound to a loopback address — so a networked deployment is authenticated out of the box. POST /api/v1/auth/login → a bearer token (12 h). Optional LDAP/AD backend (ARES_AUTH_BACKEND=ldap, needs the ldap3 package).
- **Mesh integrity & rate limiting.**
  ARES_MESH_SECRET signs every inter-node LoB / chat (HMAC-SHA256 over the content) and gates the WebSocket. A per-IP token-bucket rate limiter on /api/v1/* (tighter for /simulate & /packs/download), 429 on excess. An append-only audit log at data/audit.log (logins, device & peer changes, calibration, CoT-target & ATAK-toggle changes).
- **CoT over mutual-TLS.**
  tls:// targets with ARES_COT_TLS_CA / CERT / KEY — what a real TAK Server input expects. A CoT receive listener brings inbound GeoChat back into the chat hub.
- **Verify it works.**
  cd backend && python -m tests.test_authoritative — a 53-check harness over ITM (incl. reference pins), the ML DF, TDOA, SGP4, HF, the array interferometry, and the security pass. cd frontend && node --test tests/ — 8 pure-maths checks. CI runs both on every push.
- **Learn more.**
  http://localhost:8000/docs (interactive API) · docs/AUTHORITATIVE_v2.md (what's authoritative vs. still approximate) · docs/DEPLOYMENT.md (Jetson / laptop / Pi / cloud, air-gapped, CoT, GPS, the smoke test) · docs/BUILD_PLAN.md (the workstreams) · the source: backend/app/ (Python), frontend/src/ (React).
- **New to programming?**
  Ask the project chat for the learning roadmap — which languages and concepts (Python · JavaScript/React · web/HTTP/JSON · the RF / DF / SDR / GIS / TAK domain · the maths) to study, in what order, and which track to pick.